TNG TECHNOLOGY CONSULTING

Eliminate hidden exceptions with railway-oriented programming (ROP)

Swiss Python Summit 2026

Christoph Gietl

2026-10-22

Rapperswil SG

Table of contents

TNG TECHNOLOGY CONSULTING

1. The problem: hidden domain exceptions
2. The solution: railway-oriented programming (ROP)
3. Three ROP libraries, two architectures
4. ROP in practice with `trcks`
5. When (not) to use ROP
6. From hidden exceptions to visible failures



1 The problem: hidden domain exceptions

The problem: hidden domain exceptions

Your first day of work at a video streaming platform

```python
# Task: Find the subscription fee of `jane_doe@provider.com`!

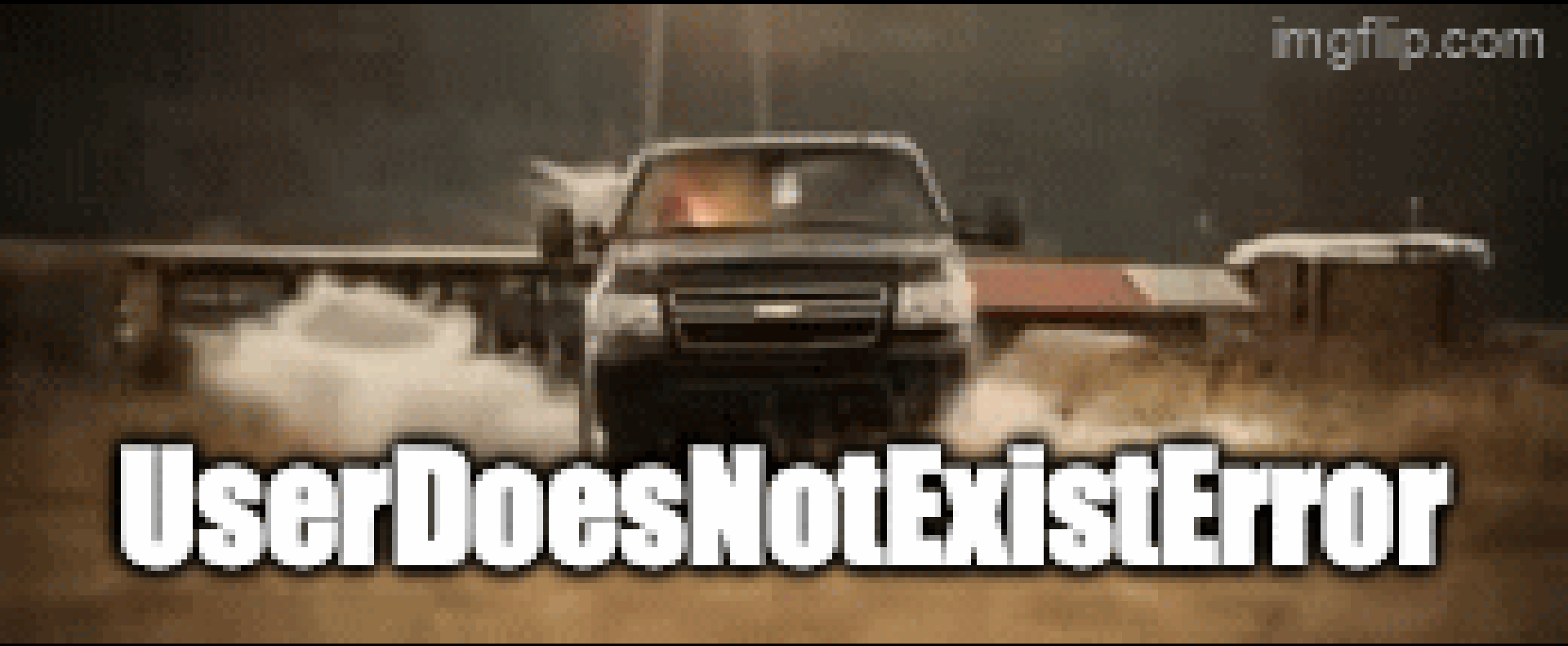
# 1. You search the codebase and find these functions:
def get_subscription_fee(subscription_id: int) -> float: ...
def get_subscription_id(user_id: int) -> int: ...
def get_user_id(user_email: str) -> int: ...

# 2. You implement this function:
def get_subscription_fee_by_email(user_email: str) -> float:
    user_id = get_user_id(user_email)
    subscription_id = get_subscription_id(user_id)
    return get_subscription_fee(subscription_id)

# 3. The type checker passes.

# 4. You call the new function:
get_subscription_fee_by_email("jane_doe@provider.com")
```




imgflip.com
UserDoesNotExistError

The problem: hidden domain exceptions

Why did the type checker not warn you?

- In Python, **exceptions are not part of the function signature**.
- Exceptions can be documented in the docstring. But **type checkers do not validate docstrings**.

What does that mean for us?

- Exceptions are an **invisible second exit path for functions**.
- The type checker can only see the happy path.

The problem: hidden domain exceptions

How does traditional Python expect us to deal with this?

Strategy	Feasibility
Read the implementation of every function you call. And the functions they call. And so on.	Not feasible. Contradicts modularity.
Use docstrings to document all domain exceptions raised or propagated by each function you write. Always keep these docstrings up-to-date.	Hardly doable. Docstrings rot.

2 | The solution: railway-oriented programming (ROP)

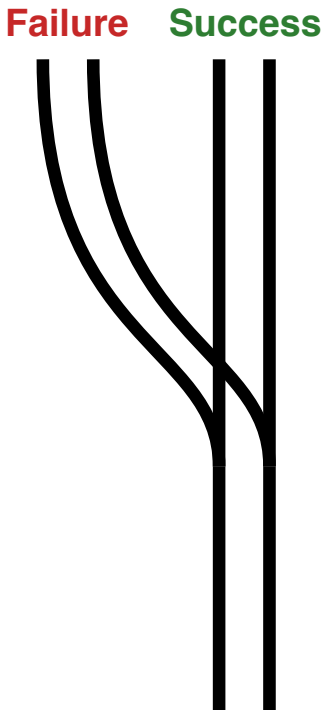
The solution: railway-oriented programming (ROP)

Where does the term ROP come form?

- comes from the F# community
- introduced by Scott Wlaschin on his blog fsharpforfunandprofit.com

What is the key idea of ROP?

- Give functions **two visible exit paths** (called **tracks**): `Success` and `Failure`.
- Use a generic return type called `Result` to encode both tracks.
- Explicitly restrict further processing to one of the two tracks.



The solution: railway-oriented programming (ROP)

How can we implement the `Result` type?

- based on classes
- as a discriminated union based on `tuple`:

```python
from typing import Literal

type Failure[F] = tuple[Literal["failure"], F]
type Success[S] = tuple[Literal["success"], S]
type Result[F, S] = Failure[F] | Success[S]

# Examples:
res_0: Result[str, int] = ("failure", "User does not exist")
res_1: Result[str, int] = ("success", 1)
```

What does `Result[F, S]` mean?

1. Value must be a `tuple` with exactly two elements.
2. First element must equal `"failure"` or `"success"`.
3. If first element equals `"failure"`, second element must be an `F`.
4. If first element equals `"success"`, second element must be an `S`.

The solution: railway-oriented programming (ROP)

How can we use `Result` as a return type?

```
type UserDoesNotExist = Literal["User does not exist"]
type UserDoesNotHaveASubscription = Literal["User does not have a subscription"]

def get_user_id(user_email: str) → Result[UserDoesNotExist, int]:
    if user_email == "erika.mustermann@domain.org":
        return "success", 1  # short form of `return ("success", 1)`
    return "failure", "User does not exist"

def get_subscription_id(user_id: int) → Result[
        UserDoesNotHaveASubscription, int
]:
    if user_id == 1:
        return "success", 42
    return "failure", "User does not have a subscription"

def get_subscription_fee(subscription_id: int) → float:
    return subscription_id * 0.1
```

What have we won?

- The **function signatures now contain the domain exceptions** *User does not exist* and *User does not have a subscription*.
- The type checker forces function callers to handle the domain exceptions.

The solution: railway-oriented programming (ROP)

Which challenges do we face?

1. How can we combine `Result` with containers (e.g. `tuple`, `list`, `dict`)?
2. How can we combine `Result` with asynchronous code?
3. How can we compose `Result`-returning functions with each other and with "regular functions"?

```python
# We want to implement:
def get_subscription_fee_by_email(user_email: str) → Result[UserDoesNotExist | UserDoesNotHaveASubscription, float]: ...

# But the output of `get_user_id` no longer matches input of `get_subscription_id`
# and the output of `get_subscription_id` no longer matches input of `get_subscription_fee`:
def get_user_id(user_email: str) → Result[UserDoesNotExist, int]: ...
def get_subscription_id(user_id: int) → Result[UserDoesNotHaveASubscription, int]: ...
def get_subscription_fee(subscription_id: int) → float: ...
```

The solution: railway-oriented programming (ROP)

Could we just compose `Result`-returning functions by hand?

```python
def get_subscription_fee_by_email(user_email: str) -> Result[UserDoesNotExist | UserDoesNotHaveASubscription, float]:
    # Apply `get_user_id` and return possible `Failure`:
    user_id_result = get_user_id(user_email)
    if user_id_result[0] == "failure":
        return user_id_result                    # `user_id_result` has type `Failure[UserDoesNotExist]`.
    user_id = user_id_result[1]                  # `user_id_result` has type `Success[int]`.
    # Apply `get_subscription_id` and return possible `Failure`:
    subscription_id_result = get_subscription_id(user_id)
    if subscription_id_result[0] == "failure":
        return subscription_id_result            # `subscription_id_result` has type `Failure[UserDoesNotHaveASubscription]`.
    subscription_id = subscription_id_result[1] # `subscription_id_result` has type `Success[int]`.
    # Apply `get_subscription_fee`:
    subscription_fee = get_subscription_fee(subscription_id)
    # Wrap output in `Success` and return it:
    return "success", subscription_fee
```

Yes, we could. But it would be ugly. And it would massively bloat our code with repetitive `if`-`return` blocks.

3 | Three ROP libraries, two architectures

Three ROP libraries, two architectures

Library	Author	License	Status	Supported type checkers	Implementation of `Result`	OOP-style function composition	FP-style function composition
Expression	Dag Brattli	MIT	stable	`mypy`, `pyright`	union of classes `Error` and `OK`	methods of `Error` and `OK`	separate functions
returns	Nikita Sobolev (drylabs)	BSD-3-Clause	beta	`mypy` (requires `mypy` -plugin)	union of classes `Failure` and `Success`	methods of `Failure` and `Success`	separate functions
trcks	Christoph Gietl	MIT	alpha	`mypy`, `pyrefly`, `pyright`	discriminated union based on `tuple`	separate wrapper class	separate functions

4 | ROP in practice with trcks

ROP in practice with `trcks`

Function composition based on method chaining with `trcks.oop`

```python
from trcks import Result
from trcks.oop import Wrapper

def get_user_id(user_email: str) -> Result[UserDoesNotExist, int]: ...
def get_subscription_id(user_id: int) -> Result[UserDoesNotHaveASubscription, int]: ...
def get_subscription_fee(subscription_id: int) -> float: ...

def get_subscription_fee_by_email(
    user_email: str,
) -> Result[UserDoesNotExist | UserDoesNotHaveASubscription, float]:
    return (
        Wrapper(core=user_email)                    # wraps the input value
        .map_to_result(get_user_id)                 # applies `Result`-returning function (resulting in two-track split)
        .map_success_to_result(get_subscription_id) # applies another `Result`-returning function if on success track
        .map_success(get_subscription_fee)          # applies "regular function" if on success track
        .core                                       # unwrap value (with type inferred precisely by type checker)
    )
```

ROP in practice with `trcks`

Composing asynchronous functions with `trcks.oop`

```python
from trcks import Result
from trcks.oop import Wrapper

async def get_user_id_from_db(user_email: str) -> Result[UserDoesNotExist, int]: ...
async def get_subscription_id_from_db(user_id: int) -> Result[UserDoesNotHaveASubscription, int]: ...
def get_subscription_fee(subscription_id: int) -> float: ...

async def get_subscription_fee_by_email(
    user_email: str,
) -> Result[UserDoesNotExist | UserDoesNotHaveASubscription, float]:
    return await (
        Wrapper(core=user_email)
        .map_to_awaitable_result(get_user_id_from_db)                  #  method name includes "awaitable"
        .map_success_to_awaitable_result(get_subscription_id_from_db)  #  method name includes "awaitable"
        .map_success(get_subscription_fee)
        .core     # inferred as `Awaitable[Result[UserDoesNotExist | UserDoesNotHaveASubscription, float]]`
    )
```

ROP in practice with `trcks`

Function composition based on higher-order functions with `trcks.fp`

```python
from trcks import Result
from trcks.fp.composition import pipe
from trcks.fp.monads import result as r

def get_user_id(user_email: str) -> Result[UserDoesNotExist, int]: ...
def get_subscription_id(user_id: int) -> Result[UserDoesNotHaveASubscription, int]: ...
def get_subscription_fee(subscription_id: int) -> float: ...

def get_subscription_fee_by_email(
    user_email: str,
) -> Result[UserDoesNotExist | UserDoesNotHaveASubscription, float]:
    return pipe(
        user_email,                                  # str
        get_user_id,                                 # str -> Result[UserDoesNotExist, int]
        r.map_success_to_result(get_subscription_id), # Result[F, int] -> Result[F | UserDoesNotHaveASubscription, int]
        r.map_success(get_subscription_fee),         # Result[F, int] -> Result[F, float]
    )
```

ROP in practice with `trcks`

Working with code that raises domain exceptions

- Wrap it once, at the boundary:

```python
def parse_amount(raw: str) → Result[Literal["Not a number"], float]:
    try:
        return "success", float(raw)
    except ValueError:  # Be specific about what to catch!
        return "failure", "Not a number"
```

- Two demo applications illustrate this pattern:
  - github.com/christophgietl/trcks-example-cyclopts: CLI tool for data transformation (ETL)
  - github.com/christophgietl/trcks-example-fastapi: REST API for subscription management

5 | When (not) to use ROP

When (not) to use ROP

Should I replace all exceptions by ROP failures? No.

Domain errors → `Result`

- expected, part of the business logic
- "user does not exist"
- "user has no subscription"
- caller must decide how to react

Bugs & panics → exceptions

- unexpected, unrecoverable
- `KeyboardInterrupt`, `MemoryError`
- assertion violations
- let them crash — do not wrap them

6 | From hidden exceptions to visible failures

From hidden exceptions to visible failures

Key takeaways

1. **Exceptions are invisible** to static type checkers. Hidden control flow causes hidden bugs.
2. `Result` puts **domain errors into the signature**. The type checker forces callers to handle them.
3. Domain errors ride the failure track, **bugs still raise**. ROP complements exceptions instead of replacing them.
4. Pick the library that fits **your team** and type checker.

How can you get started?

Wrap **one** raising function at **one** boundary of your codebase. The type checker will guide you from there.